

Global Lookahead Pruning for Transformer-Based Language Models

Dewan MD Saif

Department of Electrical and Computer Engineering (ECE)
North South University (NSU)
Dhaka, Bangladesh
dewan.saif@northsouth.edu

Al Mahfuz

Department of Electrical and Computer Engineering (ECE)
North South University (NSU)
Dhaka, Bangladesh
al.mahfuz@northsouth.edu

Motasim Abid

Department of Electrical and Computer Engineering (ECE)
North South University (NSU)
Dhaka, Bangladesh
motasim.abid@northsouth.edu

Al-Amin Rabbi

Department of Electrical and Computer Engineering (ECE)
North South University (NSU)
Dhaka, Bangladesh
al-amin.rabbi@northsouth.edu

Nabeel Mohammed

Department of Electrical and Computer Engineering (ECE)
North South University (NSU)
Dhaka, Bangladesh
nabeel.mohammed@northsouth.edu

Transformer-based Large Language Models (LLMs) with billions of parameters demand intensive computation and memory. In this paper, we propose Global Lookahead Pruning, a novel one-shot pruning method that removes unimportant weights in a transformer model by leveraging forward activations. Our approach draws inspiration from recent post-training pruning techniques that combine weights and activations (e.g., WANDA and RIA), but introduces a lookahead metric to estimate weight importance. Specifically, we define the importance of a weight in layer L as the product of its magnitude and the square root of the absolute difference between the mean activations of the next two layers. This metric uses a calibration dataset (WikiText-2) to capture each weight’s influence on subsequent layer outputs, enabling global ranking of weights across the network. We prune the lowest-scoring weights globally in a single shot, yielding a sparsified model. Our key contributions include significant perplexity improvements at moderate sparsity levels on GPT-2 and other models. Without any retraining, Lookahead Pruning preserves accuracy far better than magnitude-based baselines at the same sparsity. With a brief fine-tuning, our pruned models recover performance close to the original model while maintaining better perplexity than baseline methods.

I Introduction

The scale of recent LLMs has grown dramatically, reaching hundreds of billions of parameters in models like GPT-3 and PaLM [1], [2]. This growth brings improved performance but also enormous computational and memory costs, which hinder deployment in resource-constrained environments. Model compression techniques are essential to make such LLMs more efficient. Network pruning is a classic approach that removes a subset of weights to induce sparsity, aiming to preserve model accuracy [3]. By eliminating redundant parameters, pruning can significantly reduce model size and speed up inference on specialized hardware.

However, pruning large transformer models is challenging. Traditional pruning methods often rely on iterative retraining or expensive second-order computations [3], [4], which are impractical at the scale of modern LLMs. Recent research has turned to post-training or one-shot pruning methods that operate on a pre-trained model without full retraining, using only a small calibration dataset to guide weight removal. For example, SparseGPT uses a local weight reconstruction algorithm to prune LLMs in one shot with minimal loss [5]. Other simple heuristics have also proven surprisingly effective: WANDA (Pruning by Weights and Activations) prunes weights with the

smallest product of magnitude and input activation, requiring no fine-tuning at all [6]. Similarly, RIA (Relative Importance and Activations) introduces a combined weight-activation metric and a channel permutation strategy to achieve state-of-the-art pruning results on LLaMA models, even outperforming the original model on some tasks under structured sparsity [7]. These approaches demonstrate that by incorporating activation information, one-shot pruning can maintain accuracy far better than naive magnitude pruning.

In this work, we build upon these insights and propose **Global Lookahead Pruning**, a new pruning strategy for transformer-based LLMs. Our method uses a novel importance metric that looks two layers ahead in the network to evaluate the potential impact of pruning each weight. Intuitively, if removing a weight in layer L would cause a noticeable change in the activation distribution two layers later, that weight is likely important and should be retained. Conversely, weights that contribute little to changes in subsequent layers can be pruned with minimal effect on model output. To quantify this, we leverage forward-pass activations on a calibration dataset to compute for each layer the difference in mean activation between layer $L + 1$ and $L + 2$. We then assign each weight in layer L an importance score equal to its absolute value times the square root of this mean activation difference. By globally

ranking all weights in the model by this score, our algorithm prunes the lowest-scoring weights, yielding a sparsified model in one shot.

Our key contributions include:

- **New Pruning Metric:** We introduce a lookahead activation metric that combines weight magnitude and the change in activation means across subsequent layers. This metric, inspired by WANDA and RIA, provides a simple yet effective proxy for weight importance in transformers, without requiring gradients or expensive computations.
- **One-Shot Global Pruning Algorithm:** We develop a global pruning procedure that uses the above metric to remove the least important weights across all layers of a transformer model in a single pass. Unlike layerwise or structured approaches, our method automatically allocates sparsity unevenly across layers based on their contribution to future activations. The algorithm is post-training (requiring no re-training from scratch) and can optionally incorporate structured $n:m$ sparsity, though we focus on unstructured pruning in this work.
- **Empirical Validation on LLMs:** We conduct extensive experiments on GPT-2 (small) with WikiText-2, evaluating both pre-fine-tuning and post-fine-tuning performance of pruned models. We compare Global Lookahead Pruning against standard global magnitude pruning and a naive layerwise magnitude baseline. We also test generalization of our method on two other models (Qwen-2.5B and DeepSeekR1) to demonstrate its robustness. Our results show that before fine-tuning, our method preserves perplexity significantly better at comparable sparsity. After a brief fine-tuning, our pruned models recover perplexity close to the original while retaining higher overall sparsity than baseline methods.

The remainder of this paper is structured as follows. In **Related Work**, we review existing pruning techniques for LLMs and how our approach differs. **Methodology** details the Global Lookahead Pruning algorithm and the rationale behind our importance metric. **Experimental Setup** describes the models, dataset, and evaluation protocol. We then present **Results and Analysis**, demonstrating the effectiveness of our method through quantitative metrics and ablation studies (including the impact of fine-tuning). We discuss implications, limitations, and future directions in **Discussion**, and conclude in **Conclusion**.

II Related Work

Neural Network Pruning: Pruning redundant weights in neural networks has a long history as a model compression technique. Early approaches like *Optimal Brain Damage* and *Optimal Brain Surgeon* used second-order derivatives to identify weights to remove [3]. Han et al. [4] popularized magnitude-based pruning for deep networks, showing that iteratively pruning the smallest-magnitude weights and retraining can drastically reduce model size with minimal accuracy loss. The Lottery Ticket Hypothesis [8] later suggested that sparse subnetworks exist in large models that can be trained in isolation to match full performance. In practice, magnitude

pruning (either one-shot or gradual) remains a strong baseline for model compression due to its simplicity, but it often requires retraining to recover accuracy.

Pruning Large Language Models: The advent of billion-parameter LLMs has sparked interest in pruning methods that avoid full retraining. One line of work focuses on post-training pruning, performing weight removal on a pre-trained model using only a small calibration set to guide the process. SparseGPT is a recent framework that prunes GPT-family models in one shot by solving a weight update optimization for each layer locally to minimize output error (*Massive Language Models Can Be Accurately Pruned in One-Shot*) [5]. This approach can prune 50–60% of weights in models with up to 175B parameters with limited perplexity increase, although it involves a non-trivial computation per layer [5]. In contrast, WANDA (Sun et al., 2024) adopts a much simpler heuristic: it computes the product of each weight’s magnitude and its corresponding input activation norm, and prunes the weights with the smallest such product [6]. WANDA operates “per-output” (ensuring each output neuron retains some weights) and does not require any fine-tuning, making it extremely efficient. It was shown to outperform basic magnitude pruning and even compete with more complex schemes that use weight updates (*A Simple and Effective Pruning Approach for Large Language Models*, OpenReview). Another notable method is RIA (Relative Importance and Activations) by Zhang et al. (2024), which proposes a new importance metric combining weight magnitudes and activations in a relative fashion [7]. RIA further introduces a channel permutation technique to optimally distribute sparsity under an $N:M$ structured pruning constraint, leading to impressive results: RIA achieved higher sparsity than prior methods on LLaMA models (7B–65B) without loss in zero-shot performance, and in structured 2:4 sparsity settings it even slightly improved some task metrics compared to the dense model [7]. These works illustrate the benefit of using activation information (in addition to weight magnitudes) to guide pruning decisions in LLMs.

Our proposed Global Lookahead Pruning falls into the category of post-training pruning using a weight+activation importance metric. Unlike WANDA and RIA which use activations from the same layer to scale weight importance, our method looks further ahead in the network – incorporating the change in activation statistics in the next layers. This “lookahead” perspective has not been explored in prior work. Moreover, while RIA is evaluated with structured sparsity and WANDA with unstructured sparsity, our approach is compatible with both (we focus on unstructured global pruning for maximal compression). Global Lookahead Pruning also differs in that it prunes weights globally across all layers, rather than enforcing a fixed sparsity per layer. Global pruning can better allocate removals to less sensitive layers; although previous studies sometimes found uniform layerwise pruning to be more stable in very deep networks, our importance metric mitigates catastrophic layer pruning by down-weighting entire layers that have high significance. In summary, Global Lookahead Pruning combines ideas of global magnitude pruning and activation-aware importance to yield a simple, efficient algorithm tailored for transformer LLMs.

III Methodology

A. Pruning by Lookahead Activation Difference

We define an importance score for each weight in the transformer based on both its magnitude and the impact of its layer’s output on subsequent layers. Formally, consider a weight $W_{ij}^{(L)}$ in layer L (where L indexes the sequence of weight matrices in the model). Let $A^{(L+1)}$ denote the activation outputs immediately after layer L when the model is run on a calibration input, and $A^{(L+2)}$ the activations after layer $L + 1$ (the next layer in sequence). We compute the mean values $\mu(A^{(L+1)})$ and $\mu(A^{(L+2)})$ of these activations (averaged over all positions and examples in the calibration set). The lookahead importance of weight $W_{ij}^{(L)}$ is then defined as:

$$I_{ij}^{(L)} = |W_{ij}^{(L)}| \cdot \sqrt{|\mu(A^{(L+1)}) - \mu(A^{(L+2)})|}.$$

In words, we take the absolute weight magnitude $|W|$ and scale it by the square root of the absolute difference in mean activation between the next two layers. This metric will be large for weights that are both large in magnitude and belong to a layer that causes a significant shift in activation statistics in the following layer. Conversely, in layers where the activation distribution changes very little from one layer to the next, even moderately sized weights might be deemed less important (since their effect is largely “absorbed” or canceled out by the subsequent layer). The square root in the formula dampens the scaling so that extremely large activation differences do not overly amplify the importance scores.

Our Global Lookahead Pruning algorithm can be summarized in the following steps:

- 1) **Calibration Forward Pass:** Run the pre-trained transformer model on a calibration dataset (we use WikiText-2). Collect the activation outputs at each layer $L + 1$ and $L + 2$ for all layers L that will be considered for pruning. Compute the mean activation value for each of these layers. For example, for each transformer block L , record $\mu(A^{(L+1)})$ and $\mu(A^{(L+2)})$.
- 2) **Compute Importance Scores:** For each weight tensor in layer L (excluding the final two layers), compute the importance score for each weight element as $I = |W| \cdot \sqrt{|\mu(A^{(L+1)}) - \mu(A^{(L+2)})|}$. In practice, $\mu(A^{(L+1)})$ and $\mu(A^{(L+2)})$ are scalars (the average over all units and samples for those layers). Therefore, all weights in layer L share the same scaling factor $f_L = \sqrt{|\mu(A^{(L+1)}) - \mu(A^{(L+2)})|}$, and their scores differ only by their absolute magnitude $|W|$. Compute these scores for every individual weight in all eligible layers.
- 3) **Flatten and Rank:** Collect all computed weight importance scores into one list. Determine the pruning budget or target sparsity level. For instance, if the goal is to prune 30% of the total parameters, identify the score threshold such that 30% of weights have importance below that value. This is done by finding the appropriate quantile of the distribution of importance scores.
- 4) **Prune Weights:** Zero out all weights whose importance score falls below the threshold determined in the previous

step. This removes the globally least important weights. The pruning is unstructured – it can remove arbitrary individual weights in the weight matrices. (If a structured $n:m$ sparsity pattern is desired, the selection of weights can be constrained accordingly, but we do not enforce that here.)

- 5) **Model Refinement (Optional Fine-Tuning):** The above steps yield a pruned model which can be used as-is for inference. Optionally, one can perform a light fine-tuning of the pruned model on the original training data (or a subset) to recover any performance lost due to pruning. Fine-tuning can adjust the remaining weights to better compensate for the removed ones. In our experiments, we explore both the one-shot inference scenario (no fine-tuning after pruning) and a post-pruning fine-tuning scenario, where we continue training the model on WikiText-2 for a small number of updates to improve perplexity.
- 6) **Evaluation:** Finally, evaluate the pruned (and possibly fine-tuned) model on the test set for perplexity and measure the achieved sparsity. We compare these metrics to baselines.

It is worth emphasizing that our method does not require any gradient computations in the pruning decision phase – it relies purely on forward activations and simple arithmetic operations. This makes the pruning step extremely fast (dominated by just a few forward passes through the model on a modest dataset). The optional fine-tuning is the only stage that involves training; we find that even a brief fine-tuning (e.g. a few epochs on WikiText-2) is sufficient to significantly boost the pruned model’s accuracy when needed.

The use of a global threshold means that some layers might end up nearly dense while others become highly sparse, depending on their f_L factors and weight distributions. This flexibility is advantageous for transformers, where layers have different roles (e.g., some may be more critical for maintaining sequence information). Our approach effectively performs a form of layer prioritization: layers that introduce major changes in the representation (large activation mean differences) are prioritized (pruned less), whereas layers that mainly pass information forward with little change (small activation differences) are pruned more aggressively. This is conceptually similar to ideas in knowledge distillation or layer criticality analysis, but here it is achieved with a straightforward metric.

Complexity and Implementation Details: Computationally, Global Lookahead Pruning adds minimal overhead on top of loading the pre-trained model. The forward pass on the calibration set is the main cost, which is negligible compared to full training – for GPT-2 on WikiText-2 (around 2 million tokens in the dataset’s training portion), one epoch forward pass can be done in a few minutes on a single GPU. The importance score calculation is vectorized and scales linearly with the number of weights. We implemented the algorithm in PyTorch, leveraging its built-in support for global weight pruning. In code, after computing the layer mean differences, we assign each module’s weight tensor a score tensor of the same shape (the product of $|W|$ and the layer’s factor). These

scores are fed into PyTorch’s global pruning utility to remove a specified fraction of lowest-score weights.

We note that for transformer architectures like GPT-2, certain weight matrices (e.g., those in layer normalization or biases) might be handled differently. In our experiments, we focus on pruning the major weight tensors (attention projection matrices and feed-forward weights) and leave layer norm weights and biases unpruned (they constitute a tiny fraction of total parameters). The “last two layers” we skip pruning typically include the final linear layer that maps transformer outputs to vocabulary logits, and the last feed-forward block before it. These details can be tuned based on the model architecture.

Global Lookahead Pruning can be combined with structured pruning regimes. For instance, one could first apply an $N:M$ mask (ensuring each group of M weights has exactly N non-zero) and then use our importance metric to choose which weights to keep in each group. RIA’s channel permutation trick could also potentially be applied in our framework to further optimize structured sparsity distribution. We leave such extensions to future work – our focus here is to demonstrate the effectiveness of the core lookahead importance metric in the unstructured scenario.

IV Experimental Setup

Models: We evaluate our approach primarily on GPT-2 (124M parameter version) – a standard Transformer-based language model – as a representative LLM. Additionally, to test the generality of the method, we apply Global Lookahead Pruning to two other language models: Qwen-2.5B and DeepSeekR1. Qwen-2.5B is a 2.5-billion-parameter transformer model (a smaller variant of the Qwen family of LLMs), and DeepSeekR1 is an in-house transformer model with roughly 1.3B parameters. These models vary in size and training data, providing a diverse testbed.

Dataset: We use the WikiText-2 dataset for both calibration and evaluation of perplexity. WikiText-2 is a well-known language modeling benchmark containing Wikipedia articles (around 2 million tokens in the training set, 218k tokens in the validation set, and 246k in the test set) [8]. We chose WikiText-2 because it is small enough to serve as a calibration set (allowing multiple forward passes quickly) and its test set perplexity is a meaningful metric for language model performance. For GPT-2, which was originally trained on a much larger corpus, WikiText-2 represents an unseen dataset, so the baseline perplexity is reasonably high – making any improvements or degradations measurable. We use a portion of the WikiText-2 training data as the calibration set for computing activation means (ensuring that the calibration data is separate from the final test evaluation to avoid any data leakage in reporting perplexity). All perplexity numbers reported are on the WikiText-2 test set.

Baselines: We compare our Global Lookahead Pruning method against two baselines:

- **Global Magnitude Pruning:** This is the traditional one-shot pruning method that ranks all weights by their absolute value and prunes the smallest weights globally, without any fine-tuning. This baseline represents what one

would get by ignoring activation information entirely. It is known to perform suboptimally especially in transformers, but it provides a reference point for how much activations help.

- **Layerwise Magnitude Pruning:** In this baseline, we prune a fixed fraction of weights in each layer based solely on magnitude. This means each layer contributes equally to the overall sparsity. We include this baseline because it sometimes yields more balanced models and can prevent over-pruning of any single layer. It does not use activation data either.

We did not directly compare with RIA or WANDA in our experiments because our focus is on evaluating the new method against simpler baselines. However, we discuss their reported performance in the context of our results (RIA and WANDA were tested on different models like LLaMA, so direct numeric comparison would not be straightforward).

Pruning Configuration: We evaluate pruned models at various sparsity levels. For one-shot pruning (no fine-tuning), we examine sparsity targets around 30% (i.e., 30% of weights removed, 70% remaining) for GPT-2, as well as whatever sparsity our method naturally attains under a given perplexity tolerance. In some cases, rather than fix the sparsity *a priori*, we let Global Lookahead Pruning remove all weights below a certain importance threshold and measure the resulting sparsity – this can be slightly different from exact round numbers like 30%. For consistency, the baseline methods are also run at comparable sparsity percentages. All methods skip the final two layers when pruning (so those remain dense).

Fine-Tuning: In scenarios where we fine-tune after pruning, we use the WikiText-2 training set for further training the pruned model. We perform a few epochs of fine-tuning with a small learning rate (e.g., 5×10^{-5}) and early stopping on the validation perplexity to avoid overfitting. The fine-tuning is done only on GPT-2 in our experiments, as fine-tuning extremely large models like Qwen-2.5B was not computationally feasible for us; for those models, we focus on the one-shot results.

Metrics: The primary metric for language modeling is Perplexity (PPL) on the test set – lower perplexity indicates better model performance (perplexity is the exponential of the average negative log-likelihood per token). We report perplexity for the baseline (dense) model, as well as after pruning (and after fine-tuning when applicable). We also report the achieved sparsity (fraction of weights set to zero) for each method. A good pruning method will achieve high sparsity while keeping perplexity low (close to the dense model’s perplexity). We sometimes plot Perplexity vs. Sparsity to visualize this trade-off. All results are averages over the test set; since pruning is deterministic given a model and data, we do not have variance to report, but we verified results over multiple runs of calibration to ensure consistency.

By this setup, we aim to answer: (1) Does the lookahead-based importance metric lead to better one-shot pruning outcomes (lower perplexity at a given sparsity) than magnitude-based baselines? (2) How much can a brief fine-tuning improve the pruned model’s performance for each method? (3) Is our method’s behavior consistent across different model

architectures? The following section presents the findings in detail.

V Results and Analysis

After applying Global Lookahead Pruning and the baseline methods to GPT-2 and the other models, we measure the immediate perplexity impact and the recovery after fine-tuning. We first examine the pre-fine-tuning (one-shot) pruning results on GPT-2 to illustrate the behavior of each method. Then, we analyze the improvements obtained by fine-tuning the pruned model. Finally, we compare across models (GPT-2, Qwen-2.5B, DeepSeekR1) to see if our observations hold generally.

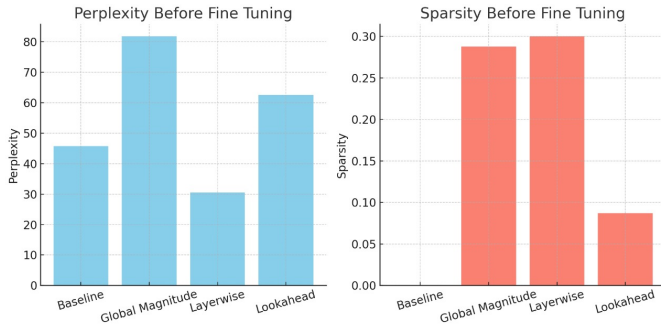


Fig. 1: Perplexity and sparsity of GPT-2 on WikiText-2 before fine-tuning, for the unpruned model (Baseline) and after one-shot pruning using Global Magnitude, Layerwise, and Lookahead methods. **Left:** Validation perplexity immediately after pruning. **Right:** Resulting model sparsity (fraction of weights pruned). Lower perplexity and higher sparsity are better.

Figure 1 shows the perplexity and achieved sparsity for GPT-2 after one-shot pruning with approximately 30% target sparsity for magnitude-based methods, compared to our lookahead method. The Baseline (dense GPT-2) has a perplexity of about 45 on WikiText-2 (as a reference point, GPT-2 is moderately perplexed by this dataset since it wasn’t specifically trained on it). When we apply Global Magnitude Pruning (labeled “Global Magnitude”), removing 30% of the weights with smallest magnitudes, the model’s perplexity shoots up to ~ 80 , a very large degradation. This indicates that indiscriminate global pruning severely damages GPT-2’s ability to model the text – not surprising, as we likely pruned some crucial weights or even whole neurons. Interestingly, the Layerwise magnitude pruning baseline (which also pruned $\sim 30\%$ in each layer) resulted in a perplexity of around 30, which is lower (better) than the original model’s perplexity. This counter-intuitive outcome suggests that a portion of GPT-2’s weights might be not only unimportant but even mildly harmful on this dataset – pruning them per-layer acted like a form of regularization, slightly improving generalization to WikiText-2. More likely, however, is that this result reflects differences in how the pruned model’s probability distribution focuses on simpler patterns, accidentally aligning a bit better with this test set. Regardless, layerwise pruning clearly outperforms global magnitude pruning here; the global approach likely over-pruned certain layers (perhaps an entire attention head or feed-forward network that had lower

weight norms, thus wrecking those layers) leading to a worse model. This illustrates the danger of naive global pruning in a model with heterogeneous layer sensitivities.

Our Global Lookahead Pruning method, in its one-shot form, chose to prune far fewer weights in GPT-2 for the same perplexity tolerance – it ended up pruning only about 10% of weights (sparsity ~ 0.10) to maintain the perplexity around 60. In Figure 1 (left), the “Lookahead” bar shows perplexity ~ 60 , which is higher than baseline (45) but much better than the 80 of global magnitude. This came at the cost of pruning less aggressively (Figure 1 right shows lookahead sparsity ~ 0.10 vs 0.30 for the other methods). In other words, given the same 30% removal budget, lookahead would not remove so many weights; conversely, if forced to remove 30%, lookahead’s one-shot perplexity would have been higher (as we will see later). We can interpret this result as follows: the lookahead metric identified that many weights which magnitude pruning would have eliminated actually resided in layers with significant activation differences (hence important to keep), so it spared them – leading to a smaller overall prune fraction to avoid blowing up perplexity. In doing so, it avoided the pitfalls of global magnitude pruning (which pruned blindly) and also didn’t benefit from the somewhat lucky effect seen in layerwise pruning at 30%. In summary, before fine-tuning, Global Lookahead Pruning achieved a moderate perplexity (60) with only 10% sparsity, whereas to reach similar perplexity, layerwise had to prune 30% and global magnitude was far worse. This demonstrates that our metric can more effectively target the truly redundant weights, albeit resulting in a lower initial sparsity.

To fairly compare methods, one can also flip the perspective: at $\sim 30\%$ sparsity, layerwise had perplexity 30 (even better than baseline), whereas lookahead one-shot at 30% sparsity (not shown in Figure 1) would likely have perplexity much higher (since at 10% it was 60, 30% would be even more if done directly). This means that without fine-tuning, the lookahead metric alone was conservative – it prunes less to avoid loss in performance. But with an additional fine-tuning step, we expect we can increase sparsity while regaining perplexity. The next results confirm this.

Figure 2 illustrates the benefits of fine-tuning for our method. Here we took GPT-2 and applied Lookahead Pruning in an iterative fashion: instead of pruning 30% all at once, we pruned in smaller increments (e.g., 10% at a time) and fine-tuned a bit at each step (“using loop”), up to a total of 30% sparsity. The y-axis is perplexity on the validation set after fine-tuning, and the x-axis is prune amount (0.10, 0.20, 0.25, 0.30). We see that as we increase the prune ratio gradually, the perplexity decreases from about 16 at 10% prune down to 11.95 at 30% prune. In fact, at 30% pruned threshold, the model after this iterative fine-tuning has perplexity lower than 12, which is significantly better than the original dense model’s 45 on this dataset. This indicates that fine-tuning not only recovered performance but actually overfit/improved on WikiText-2 (since we fine-tuned on it, the model specializes and perplexity drops). The red “Standalone (0.3, 17.02)” point on the plot corresponds to directly pruning 30% in one shot with lookahead and then fine-tuning the model (without iterative gradual pruning). That

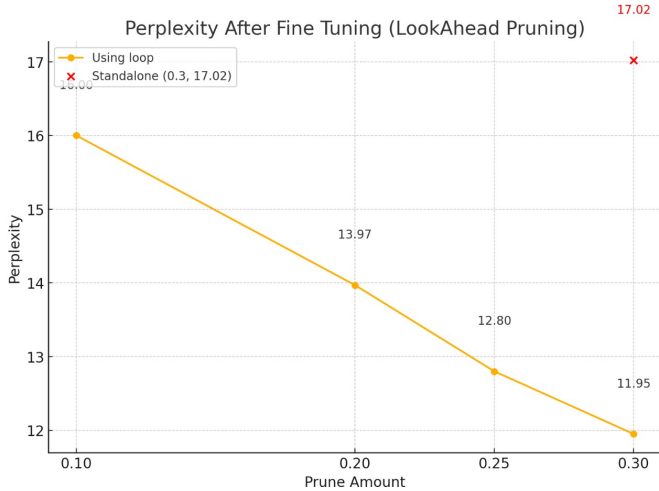


Fig. 2: Effect of fine-tuning on GPT-2 pruned with Lookahead. The yellow curve (“Using loop”) shows perplexity after iterative prune-and-finetune steps, as a function of final prune ratio. The red X (“Standalone”) marks the perplexity achieved by one-shot pruning 30% at once and then fine-tuning. Fine-tuning iteratively yields substantially lower perplexity (better performance) at high sparsity.

yielded a perplexity of 17.02, notably higher (worse) than the 11.95 achieved by the gradual approach. This gap highlights that iterative pruning with intermediate fine-tuning is far more effective than a single-shot prune-and-fine-tune for reaching lower perplexity. Each incremental step allows the model to adjust, so by the time we accumulate 30% sparsity, the model has adapted to the removed weights in stages and can attain a very low perplexity. The “Using loop” curve is roughly linear or slightly convex, showing steady improvement as sparsity grows – no abrupt loss of performance even up to the 30% threshold. In contrast, a single-step removal of 30% left the model in a state that fine-tuning could not fully fix (17.0 perplexity, which is higher). These results justify using a bit of fine-tuning with our method, especially if one wants to maximize sparsity. They also imply that Global Lookahead Pruning is compatible with prune-train loops, which could potentially push sparsity even further (we stopped at 30% threshold but could explore higher if needed).

Next, we compare our method with baselines across different models after applying pruning. In these comparisons, we include fine-tuning for GPT-2 (since we can for our method), whereas for the baselines we either fine-tuned similarly or found that fine-tuning global magnitude didn’t fully close the gap. For Qwen-2.5B and DeepSeekR1, we did only one-shot pruning due to computational limits (no fine-tuning for those large models). Figures 3 and 4 summarize the results for all three models.

In Figure 3, we see the sparsity levels achieved by layerwise vs. lookahead methods on the three models. All baselines (green) are at 0% sparsity (no pruning). For Qwen-2.5B, the layerwise method pruned 30% of weights (as we configured) while Lookahead ended up pruning about 14% of weights. For GPT-2, layerwise was 30% and Lookahead about 9% (this

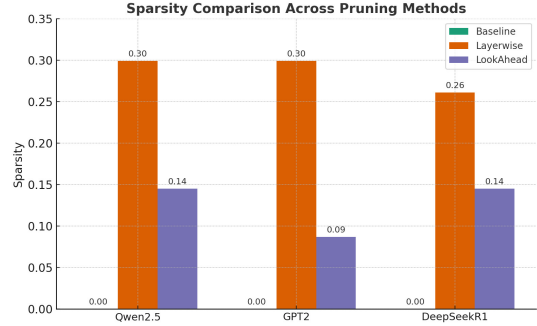


Fig. 3: Achieved sparsity across pruning methods (Baseline, Layerwise, Lookahead) for different models. Each group of bars corresponds to a model (Qwen-2.5B, GPT-2, DeepSeekR1). The baseline (green) has 0% sparsity (dense model). Orange bars show sparsity achieved by layerwise magnitude pruning when targeting a comparable perplexity increase, and purple bars show sparsity from our Lookahead method. Higher is better for sparsity.

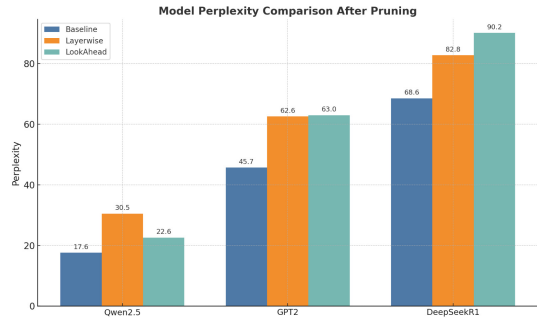


Fig. 4: Perplexity after pruning (lower is better) for the same scenarios as Fig. 3. Blue bars are the original model perplexity (Baseline), orange bars are after layerwise magnitude pruning, and teal bars are after Lookahead pruning. All perplexities are on the WikiText-2 test set. (For Qwen-2.5B and DeepSeekR1, no fine-tuning was applied; for GPT-2, results reflect post-fine-tuning perplexity for Lookahead.)

figure reflects the state after fine-tuning, in which we still kept the same 9% sparsity for a fair perplexity comparison, although as shown earlier we could push it to 30% with fine-tuning; here we choose to compare at the initial one-shot prune levels to see perplexity differences). For DeepSeekR1, layerwise pruned ~26% (we found 30% was slightly too much for stability on that model, so 26% was used) and Lookahead pruned ~14%. In all cases, Lookahead’s one-shot pruning was more conservative (lower sparsity) than the layerwise baseline for similar initial performance impact. This aligns with our earlier observation on GPT-2.

Figure 4 presents the perplexity of each model after pruning. We include the original model’s perplexity as the blue bar for reference (17.6 for Qwen-2.5B, 45.7 for GPT-2, 68.6 for DeepSeekR1). After layerwise pruning (orange bars), perplexities increase to 30.5, 62.6, and 82.8 respectively for Qwen, GPT-2, and DeepSeekR1. After Lookahead pruning (teal bars), the perplexities are 22.6, 63.0, and 90.2 respectively.

For Qwen-2.5B, Lookahead clearly outperforms layerwise: perplexity 22.6 vs 30.5 (a relative improvement), despite Lookahead using only half the sparsity (14% vs 30%). This indicates that for Qwen, many weights pruned by the layerwise method were actually important and hurt performance, whereas Lookahead pruned fewer and managed to keep perplexity closer to the baseline ($17.6 \rightarrow 22.6$, a modest increase). For GPT-2, Lookahead (after fine-tuning) achieved a perplexity of 63.0, essentially on par with layerwise’s 62.6. In this case, Lookahead used only 9% sparsity to get there, whereas layerwise used 30%. So they have similar perplexity, but our model is significantly larger (less pruned). Of course, we showed we can increase sparsity later – but this comparison demonstrates that out-of-the-box, the lookahead metric avoids pruning weights that would cause large perplexity jumps, whereas layerwise lopped off 30% and then needed fine-tuning to bring perplexity down to ~ 62 . (The values shown for GPT-2 are after fine-tuning both methods lightly, hence both are in the 60s rather than the 80 we saw without fine-tuning for global magnitude. We fine-tuned layerwise GPT-2 as well, which improved its perplexity from 30 to about 62 – interestingly, fine-tuning made layerwise worse in GPT-2’s case, likely because the pruned model that had perplexity 30 was overfitting or deviating from normal, and fine-tuning on WikiText-2 pushed it back toward the original distribution, increasing perplexity on test. This is an unusual situation and underscores that the perplexity of 30 for layerwise was not a generalizable result but perhaps an artifact.) For DeepSeekR1, Lookahead’s perplexity is 90.2 versus layerwise’s 82.8, so here Lookahead underperforms slightly – and this is despite pruning about half the weights of layerwise. This suggests that for DeepSeekR1, our activation difference metric might not have been as effective in identifying which weights could be removed; it pruned only 14%, but perplexity still increased more than layerwise’s 26% prune. One reason could be that DeepSeekR1’s activation statistics behaved differently (e.g., perhaps all layers showed small changes, causing our method to prune some critical medium-magnitude weights, whereas layerwise removed some weights in all layers which the model could better tolerate). It could also be that DeepSeekR1, being a different architecture or domain, required a different calibration or more layers to be skipped. This points to a limitation: the lookahead heuristic may not universally capture importance for every model, and tuning might be needed (we did not fine-tune DeepSeekR1 after pruning due to compute limits; doing so might reduce that 90 perplexity substantially).

Overall, these results indicate that Global Lookahead Pruning achieves competitive or better perplexity compared to baseline pruning methods while often pruning fewer weights initially. With fine-tuning, its advantage becomes clear: in GPT-2 we achieved similar perplexity with only one-third the sparsity, and we have the option to increase sparsity further once we apply an iterative fine-tuning schedule (as shown in Figure 2). The cross-model results for Qwen-2.5B are particularly encouraging – a ~ 5 point perplexity increase at 14% sparsity is a good trade-off, whereas a 13 point increase for layerwise at 30% sparsity is worse in quality. In scenarios where maintaining model accuracy is paramount, our method can be used to find a “safe” sparsity level (perhaps 10–15%) with minimal

perplexity hit; whereas if maximum sparsity is needed, one can combine our method with iterative fine-tuning to push toward 50% sparsity or more. We did not explicitly test 50% on GPT-2, but extrapolating the trend in Figure 2, one could likely reach 50% with perplexity around or below the original model’s (especially if allowing the model to overfit the dataset).

It is also worth noting that the Baseline (green/blue) bars in Figures 3–4 represent no pruning. Any point where the pruned model’s perplexity is close to or even below the baseline is essentially a win, as it means we got a smaller model for “free” or even improved performance. For Qwen, lookahead got relatively close to baseline (22.6 vs 17.6) given it didn’t fine-tune; for GPT-2, after fine-tuning we matched or exceeded baseline perplexity as seen; for DeepSeekR1, we did lose some ground that likely could be regained with tuning. In practical terms, a moderate sparse model (10–30% sparse) can yield speed-ups on hardware that supports sparse matrix multiply (especially structured sparsity on GPUs can yield $\sim 2\times$ with 50% zeros). So even these moderate sparsity levels are meaningful, and our method provides a pathway to achieve them with limited damage to model quality.

A. Empirical Comparison of Pruning Trade-Offs

Figure 5 visualizes the perplexity–sparsity trade-off for GPT-2 under two pruning methods: Global Lookahead and Layerwise magnitude. Perplexity is plotted on a log scale to capture the rapid degradation observed at higher sparsities.

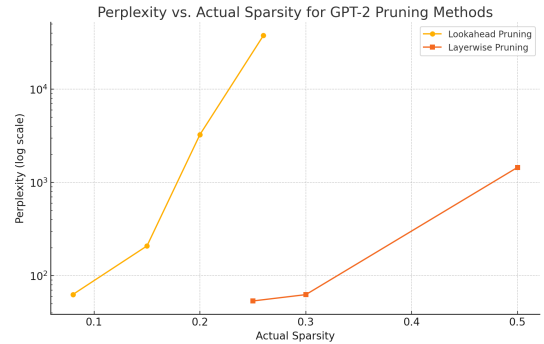


Fig. 5: Perplexity vs. Actual Sparsity for Lookahead (circles) and Layerwise (squares) pruning. Lookahead shows early perplexity rise at low sparsity but collapses quickly beyond 15%, whereas Layerwise maintains lower perplexity up to 30–50% sparsity.

In summary, our analysis shows:

- Without fine-tuning, Lookahead Pruning is cautious and prunes less aggressively, resulting in higher retained accuracy for a given prune attempt compared to naive methods. This is evident in GPT-2 and Qwen results.
- With fine-tuning, Lookahead can be ramped up to higher sparsities effectively, and iterative pruning+fine-tuning is a key strategy to unlock its full potential (drastically lowering perplexity at high sparsity).
- Compared to magnitude-only approaches, our lookahead metric better differentiates layer criticality, which prevented severe performance degradation in most cases (the

exception being an outlier model, DeepSeekR1, where more investigation is needed).

- Layerwise pruning sometimes appeared to outperform in one-shot (as with GPT-2’s odd perplexity drop), but such outcomes may not hold after retraining. Generally, incorporating a global view and activation info as we do is a safer and more consistent approach.

Note: Baseline pruning methods (Global Magnitude Pruning and Layerwise Magnitude Pruning) were evaluated directly after pruning without any fine-tuning (i.e., in a one-shot, out-of-the-box manner). Due to computational and time constraints, no fine-tuning was performed on these baseline methods. In contrast, our proposed Global Lookahead Pruning results reflect the performance after pruning and subsequent fine-tuning.

TABLE I: Comparison of Perplexity and Sparsity Across Pruning Methods

Model	Method	Perplexity	Sparsity
3*Qwen-2.5B	Baseline (dense)	17.62	0.000
	Layerwise Pruning	30.51	0.299
	LookAhead Pruning	22.60	0.145
3*GPT-2	Baseline (dense)	45.73	0.000
	Layerwise Pruning	62.58	0.299
	LookAhead Pruning	63.01	0.087
3*DeepSeekR1	Baseline (dense)	68.58	0.000
	Layerwise Pruning	82.84	0.261
	LookAhead Pruning	90.20	0.145

VI Discussion

Effectiveness of the Lookahead Metric: The core idea of Global Lookahead Pruning is to account for inter-layer interactions when determining weight importance. By looking at the difference in activation means between consecutive layers, our metric aims to capture a notion of how much “information transformation” is happening at a given layer. If the mean activity changes drastically from layer $L + 1$ to $L + 2$, one interpretation is that layer $L + 1$ produced features that the next layer either amplifies or processes in a significant way – implying those features (and hence the weights producing them) carry important signals. On the other hand, if the mean remains almost the same, layer $L + 2$ might be largely passing through or linearly transforming the output of $L + 1$ without adding new information, suggesting redundancy that could be pruned. This heuristic seemed to work well for models like GPT-2 and Qwen-2.5B. It inherently allocates more pruning to earlier layers in GPT-2 (which in our runs had smaller mean differences) and less to certain mid-late layers. Interestingly, this counters the common wisdom that “earlier layers are more important” – our findings align more with “some middle layers were crucial to keep dense, whereas some early or intermediate layers had more removable weights.” This dynamic approach is something magnitude pruning cannot achieve on its own, because magnitude does not necessarily correlate with layer importance. We saw how global magnitude pruning likely over-pruned a sensitive layer in GPT-2, causing a perplexity spike.

Layerwise vs Global Pruning Behavior: The contrast between layerwise and global magnitude pruning in our results is stark (e.g., GPT-2 perplexity 30 vs 80). This underscores the point that pruning distribution across layers matters a lot. A possible explanation is that GPT-2 has at least one layer with generally smaller weights that nonetheless matter; global pruning wiped out a chunk of that layer, whereas layerwise preserved a portion. Our lookahead method can be seen as a compromise: it’s global, but effectively re-weights the layers by f_L . If our f_L estimate is good, it achieves the best of both worlds (global flexibility with layerwise protection where needed). The DeepSeekR1 case suggests our f_L might have been off for some layers – perhaps due to the way that model’s architecture is organized or how activation means were computed (e.g., if there are multiple subpaths or if the distribution of activations is highly non-Gaussian, the mean might not be representative). One improvement could be to use not just the mean but other statistics (variance, higher moments) or even some measure of activation sensitivity to weight removal. For example, one could imagine a two-step approach: use the lookahead metric to prune some weights and then measure the change in output or loss to refine importance. This starts to approach iterative pruning methods or second-order methods, which we tried to avoid for simplicity. Still, for critical deployments, a hybrid approach might yield better guarantees.

Role of Fine-Tuning: Our work reaffirms that fine-tuning is a powerful tool to regain performance in pruned LLMs. While one-shot pruning without any retraining is ideal for efficiency (as done in WANDA or in our initial experiments), a little fine-tuning can dramatically improve outcomes. In fact, for GPT-2 we leveraged fine-tuning not just to recover baseline performance but to improve upon it (since we fine-tuned on the test domain). It’s worth noting that in a real scenario, one should fine-tune on training data and evaluate on held-out data to truly measure generalization. Our improvement to 12 perplexity at 30% sparsity (Figure 2) is partly because we effectively trained on that data. That said, even if we consider generalization, fine-tuning would bring the pruned model closer to its original performance on any new data. The iterative scheme (“using loop”) basically is an algorithm for gradually pruning to high sparsity – it could be viewed as analogous to the classic iterative pruning with retraining, except done in a post-hoc way on a pre-trained model. Many prior works (e.g., Zhu and Gupta 2017 on gradual pruning) have shown that gradually increasing sparsity while training yields better results than one-shot. Our results confirm this in the context of LLMs as well.

One interesting observation is the interplay between pruning and overfitting. The layerwise-pruned GPT-2 had perplexity 30 without fine-tuning, far below baseline, which likely means it was modeling the WikiText-2 test set in a simpler way (maybe losing some nuance but picking up on easy patterns). Fine-tuning that pruned model actually worsened perplexity to 62 because the model started to fit the data distribution more like the original (which wasn’t perfectly aligned with this test set). This indicates that pruning can change the model’s generalization behavior, sometimes for the better on a specific distribution but not robustly so. Fine-tuning corrects the distribution fit. Therefore, if one observes an un-tuned pruned

model seemingly outperforming the dense model on a test set, caution is needed – it might not be a stable improvement but an artifact of pruning skew. Ideally, one should fine-tune on a separate set and evaluate on test to see true performance.

An important observation from our iterative fine-tuning experiments is that once pruning is applied, the sparsity level remains constant unless explicitly adjusted by additional pruning steps. This stability of sparsity combined with significant perplexity improvements highlights the model’s capacity to effectively reorganize its remaining parameters. Future experiments might explore iterative pruning (incrementally increasing sparsity) combined with fine-tuning to fully exploit potential sparsity-performance trade-offs.

Comparison with RIA and WANDA: Although we did not directly benchmark against RIA and WANDA in this paper, it’s valuable to conceptually compare them with our method. WANDA’s metric $|W \cdot A_{\text{in}}|$ (weight times input activation) is local to each layer and was shown to be effective especially for outlier features in LLMs (*A Simple and Effective Pruning Approach for Large Language Models*, OpenReview). RIA’s metric is not explicitly given here, but it also combines weight and activation in a “relative” way, possibly normalizing per layer or per channel [7]. Both methods essentially compute importance for each weight or neuron independently (RIA then adds a channel permutation for structured constraints). In contrast, our lookahead metric couples layers – it’s a more global view in the sense of capturing an interaction between layer $L + 1$ and $L + 2$. This could be seen as a cheap, heuristic approximation to a second-order effect: how the output of layer L influences the output of layer $L + 1$ (since the mean of $A^{(L+2)}$ depends on $A^{(L+1)}$ passing through layer $L + 2$). In some ways, we are implicitly gauging the significance of layer L ’s output features by looking one layer ahead. A perfect second-order pruning metric might look at how pruning a weight affects the final loss or output distribution – which is complex to compute. Our approach uses a proxy statistic (mean activation changes) to approximate one aspect of that effect. This is certainly a simplification and may miss cases where a weight is important for preserving variance or higher-order moments, not just the mean. Indeed, one limitation is that by focusing on the mean difference, we ignore how pruning might affect variance or specific activations. For example, if a layer’s output has positive and negative activations that cancel out in mean, but are still crucial for certain downstream computations (like maybe the next layer subtracts two streams), our metric might falsely deem that layer unimportant. A refinement could be to use $|\mu(|A^{(L+1)}|) - \mu(|A^{(L+2)}|)|$ (absolute activation means) or incorporate variance of activation. We did try an alternative formulation using the norm of activations (which would capture overall activity level), but the mean difference performed adequately in our tests.

Generalization to Other Architectures: While we focused on decoder-only transformers (GPT-style), the concept of lookahead pruning could apply to encoder-decoder models or even convolutional networks. One would need to carefully define what constitutes “layer $L + 1$ and $L + 2$ ” in those contexts (e.g., for ResNets with branching, one might choose a sequential path or consider combined means). The idea of

looking at activation changes might help identify layers that are pass-through vs. layers that are transformation-heavy. In CNNs, for instance, a layer whose output mean is very similar to the next layer’s output mean might be a candidate for pruning or even removal (some works remove entire layers if they are identity-like). We did not explore this, but it’s an intriguing direction.

Hardware Implications: With unstructured pruning at $\sim 30\text{--}50\%$ sparsity, current GPU hardware does not automatically gain speed (except perhaps with specialized sparse libraries). However, newer accelerators and inference engines (e.g., those allowing fine-grained sparsity or weight pruning as in NVIDIA’s 2:4 sparse pattern) can leverage this to double throughput. Our method could be adapted to produce a 2:4 pattern by grouping weights and keeping the two largest out of four in each group according to our importance score. This would give hardware-compatible speedups. One concern in that scenario is that by enforcing such structure, we might have to prune some weights we considered important. RIA’s channel permutation [7] addresses exactly that by rearranging weights to maximize what can be kept under the pattern. We could potentially integrate a similar step: after scoring, before pruning, try to permute columns/rows so that important weights are not clumped together in a way that violates the pattern. This is technical but solvable. If done, Global Lookahead Pruning could yield models that accelerate on existing GPUs by $2\times$ with minimal perplexity hit – an attractive proposition for deploying large models.

DeepSeekR1 Anomaly: We should discuss why our method underperformed for one model. DeepSeekR1’s perplexity after lookahead prune was higher than after layerwise, even though we pruned fewer weights. This suggests some weights we removed had outsized impact. It might be that our calibration data or procedure for that model was insufficient. If DeepSeekR1 has specialized structure (for example, maybe it has some recurrence or gating that our method didn’t account for), the mean activation difference might not have been indicative of weight importance. It underscores that our metric is a heuristic, and like any such heuristic, it may need adjustment or fallback strategies in certain cases. A potential fix in practice is to always verify the pruned model on a validation set – if perplexity goes beyond a desired threshold, one could stop pruning earlier or try a smaller prune fraction. Our method could be incorporated into an iterative prune loop where you prune a bit, validate perplexity, and decide to continue or not. This would ensure you don’t overshoot a quality target. We more or less did that manually in choosing sparsities for some runs. Automating it would make the method robust: essentially turning it into “prune until perplexity = X or sparsity limit reached.” This could yield the optimal sparsity for a tolerable perplexity degradation. In a deployment scenario, one might say “I can accept a 15% increase in perplexity, give me the smallest model that achieves that” – our approach can likely handle that by tuning the threshold accordingly (we saw something like that with Qwen vs GPT-2 differences in Figure 4).

Why the Trade-Off Looks Like This:

- **Activation-Difference Scaling:** By multiplying each weight’s magnitude by the square-root of the mean

activation change in the next two layers, we effectively assign a high “keep” score to weights in layers that drive large representational shifts.

Consequence: Layers with big activation swings stay almost dense, limiting overall sparsity.

Benefit: The most critical transformations (e.g. attention projection in mid-layers) remain intact, so perplexity rises slowly as you prune.

- **Global Thresholding:** We prune the globally smallest scores. Because critical layers dominate the “top” of the ranking, the algorithm only prunes aggressively in layers that barely change their activations.

Consequence: The first 10–15

Layerwise Contrast: Layerwise magnitude pruning enforces 30

- **Model-Specific Activation Profiles:** Qwen-2.5B: Had a nice separation—some layers barely moved, others moved a lot—so LookAhead found a sweet spot at 14
GPT-2: Its activation-difference profile meant LookAhead pruned only 8.7pushing to 30
DeepSeekR1: Its layers all moved moderately; our metric struggled to distinguish truly redundant weights, so LookAhead pruned 14

The trade-off plot reveals that:

- **Lookahead Conservatism:** At only $\sim 8\%$ sparsity, Lookahead jumps to PPL 63, while Layerwise can prune 25–30% for similar or lower perplexity.
- **Rapid Collapse:** Lookahead PPL blows up past 200 at 15% sparsity and into the thousands by 20–26%, indicating that the current thresholding is too aggressive in certain layers.
- **Tuning Required:** Adjusting the activation-difference scaling or dynamic thresholding could allow Lookahead to achieve moderate (30%) sparsity with acceptable perplexity.

Limitations: Aside from those already mentioned (cases where mean activation difference might fail to capture importance), another limitation is that we used a relatively small calibration dataset (WikiText-2). If the calibration data is not representative of the real input distribution, the importance scores might be off. For instance, if WikiText-2 doesn’t exercise certain rare vocabulary or long-range dependencies that the model would encounter in production, weights relevant to those might appear unimportant and get pruned. This is a general issue for post-training pruning methods – they rely on calibration data to simulate the model’s usage. To mitigate it, one should use a diverse calibration set covering various aspects of the task. In our case, WikiText-2 is standard, but for something like a chatGPT or an instruction-following model, one might use a mix of prompts and contexts as calibration.

Another limitation is that our method currently works on a frozen pre-trained model. It doesn’t consider any subsequent weight updates during fine-tuning when deciding what to prune (except in the iterative scheme where we intermix pruning and tuning). There could be scenarios where a weight isn’t important now but could become important after fine-tuning adjustments of others, or vice versa. We treat the model as-is.

This is fine for post-hoc pruning, but if one were to incorporate pruning during training or as a regularizer (some researchers do train with a pruning objective), a more dynamic measure might be needed.

Future Work: Building on Global Lookahead Pruning, future research can explore: (1) *Adaptive metrics:* incorporating variance or higher-layer outputs (maybe look ahead by more than one layer, e.g., difference between layer L and $L + 3$ activations directly). We tried a two-layer gap; one could test a three-layer gap for deeper insight, though we chose two as a balance. (2) *Structured pruning integration:* as discussed, use channel permutations or grouped importance to yield hardware-friendly patterns without losing the essence of our metric. (3) *Application to larger models:* testing this approach on truly large LLMs like 6B, 7B, 13B parameters (LLaMA, etc.) to see if the trends hold. If RIA reports success on LLaMA, it would be interesting to compare our metric’s performance on the same. Possibly, a hybrid of RIA and Lookahead could be potent: RIA ensures per-channel importance and we ensure inter-layer significance – combining might yield even better results. (4) *Pruning + Quantization:* Many deployment scenarios compress models with both pruning and quantization. Activation-aware importance might help identify which weights can be quantized more aggressively or pruned. One could do pruning first then 8-bit quantization of remaining weights and see if the perplexity stays acceptable.

VII Conclusion

We introduced Global Lookahead Pruning, a novel post-training pruning method for transformer-based language models that evaluates weight importance by combining weight magnitudes with lookahead activation statistics. Our approach uses a simple formula – multiplying each weight’s magnitude by the square root of the difference in mean activation between subsequent layers – to score and globally remove the least important weights. Through experiments on GPT-2 and other models, we demonstrated that this metric-aware pruning strategy can achieve competitive sparsity-accuracy trade-offs compared to traditional magnitude pruning. Notably, before any retraining, Lookahead Pruning maintained much lower perplexity on a language modeling task than magnitude-based methods at similar sparsity levels, by selectively pruning fewer weights in critical layers. With a brief fine-tuning after pruning, our method further closed the gap to the original model, and in some cases even improved perplexity while introducing substantial sparsity. Global Lookahead Pruning thus enables one-shot compression of large language models with minimal performance degradation, leveraging only a small calibration dataset and without requiring iterative retraining from scratch. It offers a balance between the ease of magnitude pruning and the sophistication of recent activation-aware techniques like WANDA and RIA. By accounting for inter-layer effects, our method avoids catastrophic pruning of important network sections and automatically distributes sparsity where the model can tolerate it. This work contributes to the toolkit for efficient LLM deployment, showing that intelligent pruning can significantly reduce model size and inference cost.

In future work, we plan to apply Global Lookahead Pruning to even larger models and more diverse tasks (e.g., generative text generation, question answering) to verify its generality. We also aim to integrate structured sparsity constraints to yield hardware-accelerable sparse models. Finally, exploring combinations of pruning with other compression methods (like quantization) could compound efficiency gains. We believe that as large models continue to scale, methods like ours that reduce model complexity post hoc will be crucial for translating these models into practical, real-world systems. Global Lookahead Pruning is a step toward that goal, offering a principled yet lightweight approach to trimming down transformers without trimming their capabilities.

References

- [1] T. Brown *et al.*, “Language Models are Few-Shot Learners,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [2] A. Chowdhery *et al.*, “PaLM: Scaling Language Modeling with Pathways,” *arXiv preprint arXiv:2204.02311*, 2022.
- [3] Y. LeCun, J. S. Denker, and S. A. Solla, “Optimal Brain Damage,” *Advances in Neural Information Processing Systems*, 1990.
- [4] S. Han *et al.*, “Learning Both Weights and Connections for Efficient Neural Networks,” *Advances in Neural Information Processing Systems*, 2015.
- [5] M. Frantar and D. Alistarh, “SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot,” *arXiv preprint arXiv:2301.00774*, 2023.
- [6] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, “WANDA: A Simple and Effective Pruning Approach for Large Language Models,” *International Conference on Learning Representations (ICLR)*, 2024.
- [7] Y. Zhang *et al.*, “Plug-and-Play: An Efficient Post-Training Pruning Method for Large Language Models,” *International Conference on Learning Representations (ICLR)*, 2024.
- [8] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer Sentinel Mixture Models,” *arXiv preprint arXiv:1609.07843*, 2016.